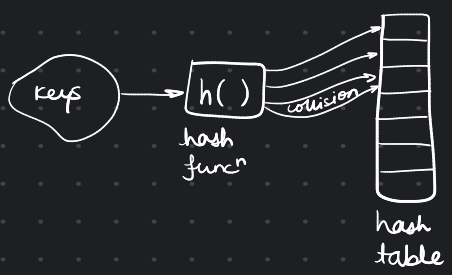
॥ श्रीगणेशाय नमः ॥

Hashing



A hash function is a mapping of a key to one of the indexes in the hash table.

Problem:
Collision: more than 1 keys can get mapped to a single index.

Some common hash functions

- Division modulo
- Mid square
- Digit extraction
- Fold boundary
- Fold shifting
- Digit truncation

Division modulo

$$h(key) = key \bmod m$$

e.g. $h(4005) = 5$
$h(3005) = 5$ } → collision
$h(2024) = 4$

Don't take a power of 2 as m. Recommended: a prime # far from any power of 2. E.g. 511 is not recommended (close to 512). 701 is okay.

N = (Binary)
→ Only last k bits are used. Not good.

If the range of the keys is high, choose a larger m based on the size of the hash table.

Mid-square method

Square the key. Choose the middle digits.

E.g. key = 54389
key^2 = 2958163321
(this or this)

Say hash table has 1000 value capacity. So h( ) would be upto 3 digits. (indices 0...999)

This uses all the digits for hash computation. Good.

Digit extraction

Say the hash table can store 1000 values (indices 0...999).

key = 5894673209
1 2 3 4 5 6 7 8 9

Choose digits at particular places.
collision high. Only a few digits are used. Not good.

Fold boundary

Say the hash table can store 1000 values (indices 0...999).

E.g. 589 624 932
→ add up
1521
→ add up
153

Again, not all digits are used for computa^n.
collision chances go up.

Fold shifting

Say the hash table can store 1000 values (indices 0...999).

key = 5 8 9 6 2 4 9 3 2

add up

2 1 4 5

219

Say 9 3 2 5 8 9 6 2 4

2 1 4 5

219

collision.

Still better than those which don't use all the digits.

Among these techniques, mid-square and fold-shifting are comparatively better.

Collision resolution techniques

- Chaining (linked list/outside/external)
- open-addressing (internal)
  - linear probing
  - quadratic probing
  - double hashing

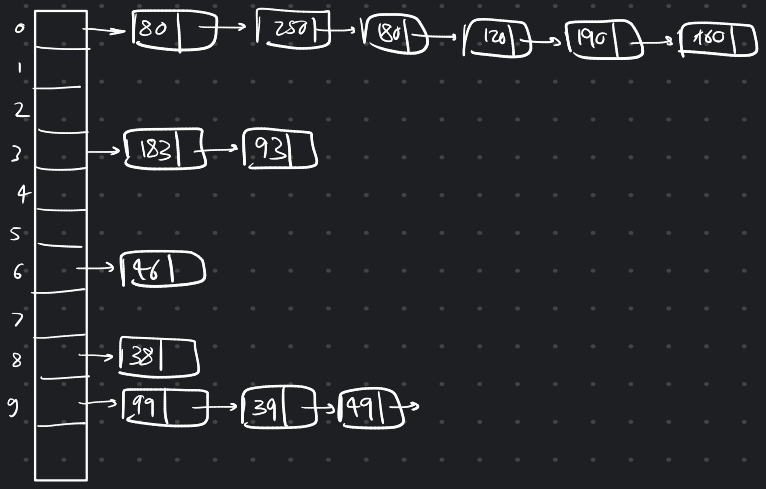
Chaining

hash-table[10] indices: 0...9

$m = 10$

$h(k) = k \bmod m$

keys: 80, 99, 46, 39, 38, 250, 180, 183, 93, 49, 66, 35, 120, 190, 60, 89, 69, 39, 62



Disadvantages:
- More space for linked lists.
  Worst case may generate n-length chains.
  Searching would take O(n) in the worst case, O(1) in the best case.
- Insertion:
  Would always happen at the beginning of the chain so O(1).
  O(n) if duplicates are not allowed.
- Deletion:
  O(1) best, O(n) worst;

Advantages:
- Can handle infinite collisions.

keys per slot = $\frac{n \text{ keys}}{m \text{ slots}} = \frac{n}{m}$

load factor $\alpha = \frac{n}{m}$

$0 \leq \alpha \leq \infty$

α can go to ∞ in chaining.

In open-addressing, load factor $\alpha = 1$.

Open addressing

Linear probing

$m = 10$

keys: 55 99 61 89 78 74 38 56 84

$h(k) = k \bmod m = k \bmod 10$

$$LP(key, i) = (h(key) + i) \bmod m$$

↳ collision #

0	89
1	61
2	38
3	
4	74
5	55
6	56
7	84
8	78
9	99

① $LP(55,0) = (h(55) + 0) \% 10 = 5 \% 10 = 5$
② $LP(99,0) = 9$
③ $LP(61,0) = 1$
④ $LP(89,0) = 9$ ✗ ← collision
$LP(89,1) = (9+1) \bmod 10 = 0$

essentially the next free slot

we check the next slots linearly until we find a free slot

⑤ 78 → 8
⑥ 74 → 4
⑦ 38 → 8 ✗
9 ✗
0 ✗
1 ✗
2 ✓

⑧ 56 → 6
⑨ 84 → 4 ✗
5 ✗
6 ✗
7 -

Disadvantages of open addressing:
- If hash table size is m, we can only put m elements in it.
- Sometimes, even if space is available, we can't put elements in.

Quadratic probing

$$QP(key, i) = (h(key) + i^2) \bmod m$$

↳ only change

keys: 55, 99, 61, 89, 78, 74, 38, 56, 84, 69

0	89
1	61
2	38
3	84
4	74
5	55
6	56
7	
8	78
9	99

① $QP(55,0) = (h(55) + 0^2) \bmod 10 = 5$
② $QP(99,0) = 9$
③ $QP(61,0) = 1$
④ $QP(89,0) = 9$ ✗
$QP(89,1) = (9+1^2) \% 10 = 0$ ✓
⑤ $QP(78,0) = 8$
⑥ $QP(74,0) = 9$
⑦ $QP(38,0) = 8$

9 → (+1²) → 12 (+2²)

⑧ 56 → 6
⑨ 84 → 4

5 → (+1²) → 8 (+2²) → 12 (+3²) = 4+9 = 13

⑩ 69
$h(69) = 9$
$(9 + 0^2) \% 10 = 9$ ✗
$(9 + 1^2) \% 10 = 0$ ✗
$2^2 = 3$ ✗
$3^2 = 8$ ✗
$4^2 = 5$ ✗
$5^2 = 4$ ✗
$6^2 = 5$ ✗
$7^2 = 8$ ✗
$8^2 = 3$ ✗
$9^2 = 0$ ✗
$10^2 = 9$ ✗
⋮

Here there's an empty slot, but regardless of how many times we try to get to it, we don't get to the empty slot for 69.

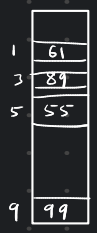
Double hashing

$$h1(k) = k \bmod m \qquad h2(k) = 1 + k \bmod (m-2)$$

$$DH(k) = (h1(k) + i \cdot h2(k)) \bmod m$$

$m = 10$

keys: 55, 99, 61, 89



① $h_1(55) = 55$ $h_2(55) = 56 \% 8 = 0$

$DH(55, 0) = 5$

② $h_1(99) = 9$

$DH(99, 0) = 9$

③ $DH(61) = 1$

④ $h_1(89) = 9$ $h_2(89) = 2$

$DH(89, 0) = 9$ ✗

$DH(89, 1) = (9 + 1 \cdot 2) \% 10 = 1$ ✗

$2 \quad (9 + 2 \cdot 2) \% 10 = 3$ ✓